

IAW · UNIDAD 5

Python: el cocinero de la casa

La misma web del Club de Lectura, cocinada en Python detrás de Nginx

HOY

Tres cosas, nada más

1

Qué cambia de PHP a Python

Una cocina distinta, el mismo camarero

2

Conectar Nginx con Python

proxy_pass: una regla parecida a la de la UD4

3

Un formulario que funcione y sea seguro

POST y el XSS de nuevo

RECORDAMOS

El restaurante de la UD4



Cliente

el navegador

Camarero

Nginx

Cocina

UD4: PHP-FPM

Hoy cambia la cocina. El camarero (Nginx) es el mismo y sigue sin cocinar.

RECUERDA — Cliente = navegador · Camarero = Nginx · Cocina = el programa que fabrica la página

LA IDEA DE HOY

Python es un cocinero de la casa

PHP (UD4)



Cada pedido trae su receta (un .php).
La lee, cocina y se olvida.
Cambias el .php → se nota al momento.

Python (hoy)



Un cocinero SIEMPRE en su cocina.
Se sabe las recetas de memoria.
Cambias el código → hay que reiniciarlo.

Si cambias el código de Python: docker compose restart app

EL ERROR N.º 1 — «He cambiado app.py y no pasa nada» → no has reiniciado la app.

LAS PIEZAS

Camarero, jefe de cocina y recetario



WSGI: el idioma en que se hablan Gunicorn y Flask. No hay que saber más.

EN EL LABORATORIO — Tres contenedores: nginx, app (Gunicorn + Flask) y cliente.

LA REGLA PARA EL CAMARERO

Lo que no está en la estantería, a la cocina

nginx/sites-available/club.conf

```
location / {
  try_files $uri @app;
}

location @app {
  proxy_pass http://app:8000;
  proxy_set_header Host $host;
}
```

¿Está en la estantería?
`try_files $uri`

...si no, a la cocina...
`@app`

...que está en app:8000.
`proxy_pass http://app:8000`

Y dile a qué web vienen.
`proxy_set_header Host`

UD4: fastcgi_pass php:9000 · UD5: proxy_pass http://app:8000

NO SE MEMORIZA — Se entiende. Está en el fichero del kit, comentada: hay que quitar las #.

DEMO · PASO 1

Sin la regla: el camarero no sabe que hay cocina

en el cliente: docker compose exec cliente sh

```
$ curl -kI https://club.iaw.local/
HTTP/1.1 404 Not Found                          ← la portada no la encuentra

$ curl -k https://club.iaw.local/hola.txt
Soy un fichero estático: lo sirve Nginx…        ← la estantería sí
```

En el navegador: https://localhost:8446

404 Not Found. Nginx solo encuentra los ficheros; de la app no sabe nada.

Y EL COMPROBADOR — bash comprobar.sh → 1 de 10 (solo el estático).

DEMO · PASO 2

Ponemos la regla: tres pasos de siempre

1



Edita

=404 → @app
y quita las # del bloque

`club.conf`

2



Comprueba

¿Está bien escrito?

`nginx -t`

3



Recarga

Sin cortar el servicio

`nginx -s reload`

en el cliente

```
$ curl -kI https://club.iaw.local/   →   HTTP/1.1 200 OK  ·  X-Cocinero: gunicorn
```

LO CONSEGUIDO — La página la cocina Python. Los estáticos los sigue dando Nginx.

¿QUIÉN ERES?

La cocina solo ve al camarero

Sin las cabeceras

/quien → «Tu IP: 172....» ← ¡es la de Nginx!
«Has entrado por: http»

Con las cabeceras

/quien → «Tu IP: 172....» ← la del cliente
«Has entrado por: https»

location @app { … }

```
proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;   # quién es el cliente
proxy_set_header X-Forwarded-Proto $scheme;                      # si entró por https
```

¿PARA QUÉ? — Los logs de la app, bloquear a un atacante, las cookies seguras… sin ellas, todos parecen Nginx.

CUANDO ALGO FALLA

Tres errores, tres cosas que pasan en un restaurante

404



«¿Qué cocina?»

Nginx no conoce la app.
Falta la regla @app.

502



Cocina cerrada

Nadie contesta.
Puerto mal o la app parada.

405



«Eso no lo hacemos»

La ruta no acepta ese método.
Falta POST en methods.

DÓNDE MIRAR — curl -kI → logs/club_error.log (Nginx) → logs/gunicorn-errores.log (la app)

LEER EL RECETARIO

Flask: cuatro piezas

club/app.py

```python
@app.route("/recomendar", methods=["GET"])
def recomendar():
    nombre = ""
    if request.method == "POST":
        nombre = request.form.get("nombre", "")
    return render_template("recomendar.html",
                           nombre=nombre)
```

`@app.route` — Qué dirección atiende

`methods` — GET = pedir · POST = enviar

`request.form` — Lo que escribió el usuario

`render_template` — El plato: HTML con huecos {{ }}

¿Ves el fallo? El formulario manda POST… y la ruta solo acepta GET.

Y SE ARREGLA — methods=["GET", "POST"] y docker compose restart app (¡es Python!).

EL FALLO DE SEGURIDAD

XSS: Flask te protege… salvo que digas |safe

ANTES ✘ · recomendar.html

```
Gracias, {{ nombre|safe }}.
```

DESPUÉS ✔ · recomendar.html

```
Gracias, {{ nombre }}.
```

Salta la alerta

|safe = «fíate de lo que escribió el usuario»

Se ve como texto

{{ }} escapa solo: <script>

UD4: htmlspecialchars a mano · UD5: {{ }} lo hace solo. Nunca |safe con lo que escribe el usuario.

REGLA DE ORO — Nunca te fíes de lo que escribe el usuario.

TU TURNO

Cinco pasos, en parejas

1	Arranca el laboratorio	`docker compose up -d --build`
2	La regla del camarero	`club.conf: @app + bloque location @app`
3	Quién es el cliente	`X-Forwarded-For y X-Forwarded-Proto`
4	El formulario	`methods POST · quita |safe · restart app`
5	Comprueba	`bash comprobar.sh → 6 de 10`

SI ACABAS ANTES — proxy_pass a app:8001, recarga: ¿qué error sale? ¿Qué dice logs/club_error.log?

HOY

Python listo para un servidor de verdad

1

Sesiones

Que el club sepa quién eres… y que nadie se haga pasar por ti

2

Secretos

La SECRET_KEY: fuera del código

3

Depuración apagada

Que un error no enseñe tu código

4

Cuántos cocineros

Los workers de Gunicorn

AL FINAL — bash comprobar.sh → 10 de 10

La pulsera de un festival



Ana

hace login

la pulsera (cookie «session»)

```
eyJ1c3VhcmlvIjoiYW5hIn0.auIQ1Q.lrqApKvKGAGSWhgtgJvVFynHLb8
= {"usuario":"ana"}  +  fecha  +  EL SELLO
```

Se puede LEER

No va cifrada: no guardes ahí nada secreto.

No se puede FALSIFICAR

Sin el sello (la SECRET_KEY), Flask la rechaza.

EL SELLO — La SECRET_KEY. Si alguien la sabe, fabrica pulseras de quien quiera.

LA PUERTA DE LOS SOCIOS

Mirar la pulsera antes de dejar pasar

Ahora: /socios sin login

Entra cualquiera: «Hola, ¿?». Nadie mira la pulsera.

club/app.py

```
@app.route("/socios")
def socios():
    if "usuario" not in session:           # ¿no hay pulsera?
        return redirect(url_for("entrar"))   # a la puerta
```

Y DESPUÉS — docker compose restart app → sin login, a /entrar; con login (ana / club2026), «Hola, ana».

EL SELLO DEL CLUB

La SECRET_KEY: larga, al azar y fuera del código

generarla (en el contenedor app)

```
$ docker compose exec app python -c "import secrets; print(secrets.token_hex(32))"
```

config/club.env

```
SECRET_KEY=8f3c1e9a5b7d…   (64 caracteres)
```

club/app.py

```
app.secret_key = os.environ["SECRET_KEY"]
```

Nunca en el código ni en Git

Si la subes a GitHub, cualquiera fabrica pulseras de administrador.

Y DESPUÉS — docker compose restart app: el cocinero lee el entorno al empezar el turno.

UNA API

Los mismos datos, para llevar

club/app.py

```python
@app.route("/api/libros")
def api_libros():
    return jsonify(LIBROS)
```

https://localhost:8446/api/libros

```
[
  { "anio": 1950,
    "autor": "Miguel Delibes",
    "titulo": "El camino" }, …
]
```

Página = un plato en la mesa (HTML) · API = un pedido para llevar (JSON).

¿PARA QUIÉN? — Una app de móvil, otra web, un script… cualquier programa que quiera los datos.

DEPURACIÓN APAGADA

Un error no puede enseñar tu código

FLASK_DEBUG=1

/error enseña la traza con TU código y la consola del depurador: ejecutar Python en tu servidor.

FLASK_DEBUG=0

/error → «Internal Server Error».
El detalle, en el log: solo lo ves tú.

config/club.env

```
FLASK_DEBUG=0          # y después: docker compose restart app
```

COMO EN LA UD4 — display_errors = Off: el error, al log; al usuario, una página genérica.

CUÁNTOS COCINEROS

Los workers de Gunicorn

workers = 1 (de fábrica)

Un solo cocinero: si un pedido tarda, todos esperan.

workers = 2 × CPU + 1

Servidor con 2 CPU → 5 cocineros a la vez.

config/gunicorn.conf.py · ver los procesos: docker compose exec app ps -o pid,args -C gunicorn

```
workers = 5          # 2 CPU → 2 × 2 + 1
timeout = 30         # segundos antes de dar por perdido un pedido
```

COMO EN LA UD4 — pm.max_children: la misma idea (cuántos a la vez), otra fórmula.

LABORATORIO GUIADO

Cuatro arreglos → 10 de 10

1	La puerta de /socios	`club/app.py: if "usuario" not in session`
2	El sello	`club.env: SECRET_KEY de 64 caracteres`
3	Depuración apagada	`club.env: FLASK_DEBUG=0`
4	Los cocineros	`gunicorn.conf.py: workers = 5`
5	Reinicia y comprueba	`restart app → bash comprobar.sh → 10/10`

SI ACABAS ANTES — Copia tu cookie «session» y decodifícala: echo eyJ… | base64 -d. ¿Qué se ve?

LO QUE NOS LLEVAMOS DE LA UD5

Resumen de puntos clave

- Python = cocinero de la casa: cambias código → restart app
- try_files $uri @app + proxy_pass http://app:8000
- X-Forwarded-For y -Proto: quién es el cliente de verdad
- {{ }} escapa solo: nunca |safe con datos del usuario
- Sesión = pulsera con sello (SECRET_KEY, fuera del código)
- FLASK_DEBUG=0 · workers = 2 × CPU + 1

El camarero sirve lo de la estantería; lo demás, al cocinero de la casa.

LO SIGUIENTE

UD6 · Los datos, en una base de datos

- Los libros, en MariaDB
- Python y PHP leyendo de la misma base
- Y todo, en la Práctica 2